

Mind Space — Implementation Plan

Document 06 of 06 | Version 1.0 | PFE Project — 15 Days

One rule: the GOAL of the day is the only thing that matters. If you hit the goal but skipped half the tasks, you are on track. If you completed all tasks but missed the goal, you are not.

Phase 1 — Foundation (Days 1–3)

Day 1 — Project skeleton + Supabase connected

Goal:

Flutter app runs on device and reads a row from Supabase.

- Create Supabase project. Save URL + anon key in .env file. Never commit.
- flutter create mind_space. Add: supabase_flutter, flutter_riverpod, go_router, flutter_dotenv, google_fonts.
- Run app on physical Android device (not emulator). Must compile clean.
- Create test table in Supabase → insert one row by hand → read it from Flutter → print(). Delete table.
- Draw the system architecture on paper: Flutter → Edge Functions → Postgres → LLM/Embed APIs. Photograph for thesis.
- DONE when: app runs on device, prints data fetched from Supabase.

Day 2 — Auth + profiles + RLS

Goal:

A user can sign in. Their profile exists. Another account cannot see their data.

- Enable Supabase Auth: Google OAuth + email magic link. Test both from Supabase dashboard before touching Flutter.
- Build auth screen in Flutter (no design polish yet — just functional).
- Create profiles table. Add handle_new_user trigger (see Document 05).
- Enable RLS on profiles. Write select + update policies.
- TEST: sign in with account A, sign in with account B, confirm B cannot read A's profile row.
- DONE when: sign in works on device, profile row auto-created, RLS proven with two accounts.

Day 3 — First Edge Function + first LLM call

Goal:

Flutter → Edge Function → Groq → Flutter round trip working.

- Install Supabase CLI. Write hello-world Edge Function. Deploy. Call from Flutter. Print response.

- Get Groq API key (console.groq.com, free, no card). Store as Edge Function secret.
- Upgrade Edge Function: accept a prompt, call llama-3.1-8b-instant, return reply. Test with curl first.
- Build temporary test screen in Flutter: text field + button → LLM reply appears. Celebrate.
- DONE when: round trip works. Flutter tap → LLM text on screen.

Phase 2 — AI pipeline (Days 4–7)

Day 4 — Chat schema + streaming chat function

Goal:

Real streaming conversation with Sage. History persisted.

- Create chats and messages tables (Document 05). Enable RLS on both.
- Upgrade Edge Function to chat-stream: save user message → fetch history → stream Groq reply via SSE → save assistant message.
- Flutter: learn to consume SSE stream. Pattern: Stream<String> listened to and appended to Riverpod state.
- Build rough chat UI: message bubbles, input box, send button. Design polish comes Day 12.
- DONE when: send a message, watch Sage stream a reply token by token. Close app, reopen, history is there.

Day 5 — Sage system prompt (most important day)

Goal:

Sage talks the way the product needs. EFT-informed. Non-prescriptive. Warm.

- Read Anthropic prompt engineering guide (docs.anthropic.com/prompting) end-to-end.
- Write Sage system prompt v1. Core requirements: (1) listen before reflecting, (2) identify EFT emotion layer underneath surface, (3) never advise, (4) use user's own language, (5) respond aware of Arc processing_stage.
- Include cross-arc context injection slot: "Background: other things currently in this person's life: [injected here]."
- Test with 10 hand-written messages: work stress, family conflict, hope, exhaustion, anger, indecision, crisis-edge, joy, grief, numbness. Read every reply out loud.
- Iterate prompt 3–5 times. Save each version as sage_v1.txt, sage_v2.txt. This is thesis evidence.
- DONE when: 3+ versions saved with notes. Sage voice feels like the brief — short, listening, not advising, EFT-aware.

Day 6 — Reflection generator (structured output)

Goal:

end-chat generates and saves a structured reflection. Reflection screen shows it.

- Create reflections and emotion_spirits tables (Document 05). Seed 6 spirits.

- Write end-chat Edge Function: fetch conversation → call Groq with response_format: json_object → parse JSON → save reflection.
- Reflection prompt: hidden chain-of-thought (identify EFT layer internally) → output { spirit_id, what_sage_heard, question_to_sit_with, shared_perspective }. The what_sage_heard must reflect the LAYER underneath, not the surface label.
- Add cross-arc context injection to reflection prompt: fetch last what_sage_heard from 2–3 other active arcs, prepend as background context.
- Build reflection screen in Flutter (rough). Show spirit, text, question.
- DONE when: tap Wrap Up after a real chat, see reflection card, row exists in DB with correct EFT-layered text.

Day 7 — Safety classifier + crisis flow

Goal:

Crisis signals handled correctly. Provably.

- Add safety-check Edge Function: pre-message classification. Prompt: "Classify as SAFE or CRISIS. One word only." Use llama-3.1-8b-instant (fast, free, 14400 RPD).
- If CRISIS: return pre-written response (not LLM-generated) + flag. Flutter shows crisis resources card. Message saved with flagged_for_safety = true.
- Test 15 messages: 5 clear-safe, 5 distressed-but-safe, 5 crisis-signal. Tune prompt until: 0 false negatives, <2 false positives. Document results.
- Build in-app crisis UI: calm card, resources (Algeria: SAMU 15 / Oran emergency resources), "I'm safe, continue" button.
- Safety must fail CLOSED: if API is down, block chat and show resources. Never fail open.
- DONE when: crisis flow demo works. You can show a committee member exactly what happens.

Phase 3 — The Arc system (Days 8–10)

Day 8 — Embeddings + pgvector

Goal:

Reflections are embedded. Similarity queries return intuitive results.

- Enable pgvector in Supabase dashboard. Add vector(1536) column to reflections table.
- Get OpenAI API key (\$0/free trial credits sufficient for PFE). Store as Edge Function secret.
- Write embed-text Edge Function: text in → OpenAI text-embedding-3-small → 1536-dim vector out.
- Wire into end-chat: after saving reflection, call embed-text, save vector to embedding column.
- Open Supabase SQL Editor. Write query: given reflection_id, return 5 most similar reflections using <=> operator. Run on real data. Do the results make sense?
- DONE when: reflections table has populated embedding column. Similarity query returns intuitive results.

Day 9 — Arc assignment algorithm

Goal:

New reflections get assigned to Arcs. The algorithm is explainable.

- Create arcs table (Document 05).
- Write assign-arc Edge Function implementing the full algorithm:
 1. User has 0 arcs → create new arc, done.
 2. Compute cosine similarity vs. all arc centroids using pgvector <=> operator.
 3. If closest arc similarity ≥ 0.78 → assign, update centroid (running average), increment session_count, update processing_stage.
 4. If similarity 0.65–0.78 (soft zone) → assign to closest arc, set needs_review = true (internal log).
 5. If similarity < 0.65 → create new arc. Name it with llama-3.1-8b-instant (2–4 words, user perspective, no clinical language, temperature 0.3). Assign dominant spirit from reflection.
- Centroid update formula: $\text{new_centroid} = (\text{old_centroid} * \text{old_count} + \text{new_embedding}) / (\text{old_count} + 1)$
- Test: fabricate 20 reflections covering 4 themes (work, family, sleep, friendship). Run in random order. Inspect arcs. Document threshold empirically.
- DONE when: arcs form predictably. You can explain any given assignment to your binome.

Day 10 — Arc insight generation + Arc detail screen

Goal:

Users with 3+ sessions can generate a macro insight. It feels earned, not generic.

- Create arc_insights table (Document 05).
- Write generate-arc-insight Edge Function using OpenRouter deepseek/deepseek-r1:free (200 RPD sufficient):
 1. Fetch all reflections in arc, chronological order (what_sage_heard + spirit + date + processing_stage_at_creation).
 2. Include cross-arc context: last what_sage_heard from other active arcs.
 3. Prompt: structure as EFT case formulation — how it evolved (processing stages visible) + pattern noticed (recurring core emotion or belief).
 4. Implement caching: if insight generated < 7 days ago AND session_count unchanged → return cached.
- Build Arc detail screen (S7) in Flutter: header, session list, Generate Arc Analysis CTA (disabled if < 3 sessions), insight card.
- DONE when: generate insight for a real arc with 3+ sessions. Read it. It should feel like a thoughtful reflection on a journey, not a summary.

Phase 4 — Screens (Days 11–13)

Day 11 — Design system + Home + Onboarding

Goal:

Design tokens live in Flutter. Home and onboarding feel like the product.

- Implement AppTheme in Flutter: ThemeData with all color tokens from Document 04. Typography using google_fonts (Inter). Border radius constants. Spacing constants.
- Build 3 core components: MindSpaceButton (primary/secondary/destructive variants), EmotionSpirit (orb with breathing animation per spirit type), AppCard (standard card with correct bg/border).
- Build onboarding: 3 swipeable slides using PageView. Last slide: "Get started" CTA → auth.
- Build home screen: greeting, ambient spirit orb (dominant spirit of most recent reflection), open Arc cards (horizontal scroll), "Start a conversation" affordance. Empty state: orb neutral, greeting, one-liner.
- CRITICAL: design the empty state first. First-time users will see this for several days.
- DONE when: fresh install opens to a beautiful, calm, intentional home screen.

Day 12 — Chat + Reflection screens polished

Goal:

The two most important screens feel premium.

- Re-skin chat UI: user bubbles right-aligned (--bg-elevated), Sage messages left-aligned NO BUBBLE (floating text). Input bar with mic icon + text + send. "Sage is listening · No solutions yet" phase indicator.
- "Wrap Up" pill appears top-right after 4 user messages (count tracked in state).
- Re-skin reflection screen: cascade animation (spirit → what_sage_heard → question → shared_perspective → arc_update). Each section has its own entrance timing (see Document 04).
- Add micro-interactions: haptic on message send, medium haptic on spirit reveal, orb ambient breath on home.
- WARNING: Do NOT build drag-and-drop organize mode. Replace with simpler "Edit Arc" bottom sheet (M3) — already designed. Saves 1.5 days.
- DONE when: hand phone to a friend, ask them to chat for 2 minutes. Watch their face when reflection appears.

Day 13 — History + Arc grid + Settings

Goal:

Back half of app is complete. Every screen is reachable.

- History screen (S5): segmented control Timeline/Arcs. Timeline: chronological list, each row has spirit dot (small, with one-time micro-tooltip legend on first view), Arc tag pill, "View Reflection >" link. Emotion filter chips.
- Arcs grid (S6): 2-column grid. Active arcs full opacity. Archived section dimmed. Organize button → simplified list-based assign (not drag-and-drop).
- MAKE emotional journey "snake" on Arc detail (S7) use actual spirit icons (not circles). Make each icon tappable to show session summary.

- Settings (S9): account email (read-only), Sign Out, Export Data (JSON download), Delete Account (confirm dialog with real cascading delete).
- Wire up M2 (Add Past Chat Sheet) and M3 (Edit Arc Bottom Sheet).
- DONE when: every screen in Document 03 is reachable. App is feature-complete.

Phase 5 — Evaluation + Polish (Days 14–15)

Day 14 — Evaluation harness + prompt tightening

Goal:

I can prove the AI works with numbers, not vibes.

- Write 30 test conversations in a JSON file. Cover: short/long, anxious/calm/ambiguous, multiple emotions, crisis-edge, Arabic-language messages (if app accepts Arabic).
- Run all 30 through end-chat pipeline. Save results to CSV.
- Grade each reflection 0–2 on 5 dimensions: warmth, accuracy, EFT-layer depth (did it get past the surface?), non-prescriptiveness, brevity. Compute per-dimension averages. This is your headline table.
- Also test arc assignment on all 30: grade each assignment as correct/soft-zone/incorrect. Compute accuracy. Document threshold calibration.
- Identify bottom 5 reflections. For each: write why it scored low. Modify prompt to address top 2 failure modes. Re-run. Compare before/after. CHART THIS.
- DONE when: you have a table with before/after quality scores. That table goes in your thesis defense.

Day 15 — Device testing + demo prep + fallback

Goal:

I will not get embarrassed at graduation.

- Install app on 3 different Android devices. Use for 1 hour each. Note every bug and every awkward loading state.
- Fix top 5 issues. Explicitly ignore the rest.
- Write 5-minute demo script: Onboarding → first chat → reflection → second chat (different topic) → third chat (same topic as first, watch Arc form) → Arcs grid → generate insight. Practice 3 times. Time it.
- Build offline fallback demo mode: hardcoded scripted conversation behind a kDebugMode flag. If wifi dies at defense, flip flag and run canned demo with pre-generated content.
- Ensure all debug label bars (H-xx labels) are behind kDebugMode. Production build shows none of them.
- Draft thesis technical chapter sections: problem → mental model → architecture diagram → Arc algorithm + math → EFT prompt engineering + versions → evaluation results + graph → limitations + future work.
- DONE when: you could lose your charger 1 hour before defense and still nail the presentation.

What to cut if you fall behind

Cut this	Keep this — never cut
Settings beyond sign-out	The reflection card (S4) — this is the product
Arc organize mode / drag-and-drop	The Arc assignment algorithm — this is the thesis
Onboarding (skip to auth if needed)	The safety classifier — non-negotiable
Arc insight generation (keep Arcs without macro insight)	Cross-arc context injection — differentiator
Day 14 full eval → do 10 reflections instead of 30	The offline fallback demo mode
Emotion filter in Timeline	EFT emotion layer in reflections
Export data feature	Delete account (legally required in most regions)

API cost estimate for PFE

Service	Usage estimate	Cost
Groq (llama-3.3-70b)	~500 chat + reflection calls	Free tier (≤1000 RPD)
Groq (llama-3.1-8b)	~500 safety + naming calls	Free tier (≤14400 RPD)
OpenRouter (deepseek-r1)	~50 arc insight calls	Free tier (≤200 RPD)
OpenAI (text-embedding-3-small)	~600 embedding calls × avg 50 tokens	~\$0.001 total
Supabase	Database + Edge Functions	Free tier (500MB DB, 500K function calls/month)
Total estimated cost		< \$1 USD for entire PFE build